

GroupFunctions.jl: computing individual entries of the irreducible representations of the unitary group $U(d)$

David Amaro-Alcalá ¹ and Konrad Szymański ¹

¹ Research Centre for Quantum Information, Institute of Physics, Slovak Academy of Sciences, Dúbravská cesta 9, Bratislava, Slovakia

DOI: [N/A](#)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: 01 January 1970

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

[GroupFunctions.jl](#)¹ is a Julia library for computing individual matrix elements of irreducible representations of $U(d)$. These matrix elements, called group functions, can be evaluated symbolically or numerically. For $SU(2)$, they reduce to the Wigner D -functions. The library computes these matrix elements in a carrier-space basis enumerated by Gelfand–Tsetlin patterns ([Gelfand & Tsetlin, 1950](#)). It can also compute entire representation operators, construct input unitaries from parameterisations common in quantum optics, translate Gelfand–Tsetlin patterns into occupation-number kets, and compute the associated Schur functions. Results can be exported in a form compatible with Mathematica.

Statement of need

Representations of the unitary group $U(d)$ arise in many subfields of physics and mathematics, and computations often reduce to evaluating their matrix elements, called group functions. For an irrep labelled by λ , the corresponding object is

$$D_{\text{out,init}}^{(\lambda)}(U) = \langle \text{out} | D^{(\lambda)}(U) | \text{init} \rangle,$$

where *init* and *out* denote basis states in the representation carrier space.

In mathematics, summing the diagonal group functions gives the trace of the representation matrix of U . This trace is the character of the representation and the Schur polynomial of the eigenvalues of U , an object central to algebraic combinatorics and symmetric function theory.

In quantum physics, group functions appear in several settings. In quantum optics, a group function gives the transition amplitude of photons through a linear optical network. The same object can be used to characterise quantum devices ([Amaro-Alcalá, 2026](#)) and to describe the symmetry properties of states ([Otto & Szymański, 2024](#)). One important subproblem is boson sampling ([Aaronson & Arkhipov, 2011](#)), where the transition amplitude reduces to a permanent whose evaluation is classically computationally hard. Whether noisy real-world quantum devices can perform this computation remains an active question.

Some of these tasks require only a numerical estimate, whereas others require an exact symbolic group function. [GroupFunctions.jl](#) addresses the latter need. Although related packages exist (see below), to our knowledge, none is designed to compute individual representation-matrix entries symbolically. When applicable, computing an individual entry is more efficient than assembling the entire matrix.

¹David Amaro-Alcalá developed the package, its algorithms, and its test suite, and prepared the initial documentation. Konrad Szymański substantially revised and expanded the documentation and developed additional examples, and contributed to the standardisation of the API.

Similar software

Several existing packages relate to `GroupFunctions.jl` but address different problems. `SUNRepresentations.jl` ([QuantumKitHub contributors, 2020](#)) and the algorithm of Alex et al. (2011) (with appendix code) compute $SU(d)$ Clebsch-Gordan coefficients. `ReplAB` ([Rosset et al., 2021](#)) supports manipulating irreducible representations of various groups, including $U(d)$, but provides only indirect numerical access to group functions. `IntegrateUnitary.jl` ([Pawela & Puchała, 2026](#)) performs symbolic integration over compact groups rather than evaluating representation matrices. `haarpy` ([Cardin et al., 2024](#)) implements Weingarten-calculus methods. Other libraries focus on quantum optics. `BosonSampling.jl` ([Seron & Restivo, 2024](#)), `Perceval` ([Heurtel et al., 2023](#)), and `Q0ptCraft` ([Aguado et al., 2023](#)) numerically model linear optical devices, while `The Walrus` ([Gupt et al., 2019](#)) helps compute amplitudes for Gaussian boson sampling. These packages do not target the symbolic computation of individual representation-matrix entries, which is the primary purpose of `GroupFunctions.jl`.

Example use and documentation

The library primarily computes matrix elements of a representation between basis states, with auxiliary functions that support calculations in quantum optics. These matrix elements can be computed from a symbolic matrix. The following example evaluates a matrix element between states of the $U(10)$ symmetric irrep corresponding to 7 bosons.

```
λ = [7,0,0,0,0,0,0,0,0,0]; basis=basis_states(λ); # integer partition and basis
init = findfirst(gt -> occupation_number(gt)==[0,0,0,1,1,1,1,1,1,1],basis);
out = findfirst(gt -> occupation_number(gt)==[1,1,1,1,1,1,1,0,0,0],basis);
group_function(λ, basis[init],basis[out]) # symbolic matrix element
```

`GroupFunctions.jl` is currently single-threaded. On an AMD Ryzen 7 PRO 4750U laptop CPU, the symbolic computation in the example took approximately 4 seconds after compilation warmup, without reusing cached intermediate or final results.

This functionality supports more complex calculations. For example, the library has been used to numerically evaluate $SU(d)$ group characters in randomised benchmarking ([Amaro-Alcalá, 2026](#)). Further examples, applications, and mathematical background are available in [the documentation](#).

Design choices

This library provides a unified method for computing representation matrix elements of $U(d)$ irreps specified by integer partitions of length at most d , including computations with symbolic input matrices. Several computational routes are possible in principle. For symmetric irreps, which model fully indistinguishable bosons, one can manipulate states as polynomials of creation operators applied to the vacuum. Another approach constructs and exponentiates the Lie algebra generators in the chosen representation. Both approaches are computationally expensive. More restricted methods based on generating functions also exist ([Prakash, 1996](#)).

The author of the original package chose the Grabmeier-Kerber formula ([Grabmeier & Kerber, 1987](#)) as the most general solution. It expresses the matrix element as a sum of monomials in the entries of the input matrix, weighted by the irreducible representation and the states in question. Our implementation optimises the enumeration of the double cosets that index this sum by grouping permutations that contribute the same monomial. This implementation provides the library's main function, `group_function`.

Internally, the algorithm represents basis states as semistandard Young tableaux. The user-facing functions expose the equivalent Gelfand-Tsetlin patterns through the `GTPattern` data structure and provide utility functions for common quantum optics scenarios, such as `occupation_number`.

Availability

The library is available under the MIT licence and can be installed through the Julia package manager:

```
] add GroupFunctions
```

AI usage disclosure

OpenAI Codex (GPT-5.3) assisted with optimising the performance of the double-coset enumeration at a late stage. The authors developed the mathematical design and proof of correctness of the optimised algorithm. Anthropic Claude (Opus 4.8) assisted with code review and language review of the documentation and manuscript. The authors reviewed and edited all AI-assisted changes.

Acknowledgements

We thank Dr. Hubert de Guise for helpful discussions and suggestions on the bibliography, and Dr. Alonso Botero for suggestions that improved the presentation. Mitacs CALAREO, DeQHOST APVV-22-0570, QUAS VEGA 2/0164/25, Postdokgrant APD0161, and the Stefan Schwarz programme supported this work. David Amaro-Alcalá acknowledges the indirect support of the Government of Alberta and NSERC during his PhD studies at the University of Calgary.

References

- Aaronson, S., & Arkhipov, A. (2011). The computational complexity of linear optics. *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*, 333–342. <https://doi.org/10.1145/1993636.1993682>
- Aguado, D. G., Gimeno, V., Moyano-Fernández, J. J., & Garcia-Escartin, J. C. (2023). QOptCraft: A Python package for the design and study of linear optical quantum systems. *Computer Physics Communications*. <https://doi.org/10.1016/j.cpc.2022.108511>
- Alex, A., Kalus, M., Huckleberry, A., & Delft, J. von. (2011). A numerical algorithm for the explicit calculation of $SU(N)$ and $SL(N, \mathbb{C})$ Clebsch-Gordan coefficients. *Journal of Mathematical Physics*, 52(2), 023507. <https://doi.org/10.1063/1.3521562>
- Amaro-Alcalá, D. (2026). Kostant relation in filtered randomized benchmarking for passive bosonic devices. In *Phys. Rev. A* (Vol. 113). American Physical Society. <https://doi.org/10.1103/xmlq-vssg>
- Cardin, Y., Guise, H. de, & Quesada, N. (2024). Haarpy, a Python library for Weingarten calculus and integration of classical compact groups and ensembles. In *GitHub repository* (0.0.6). <https://github.com/polyquantique/haarpy>; GitHub.
- Gelfand, I. M., & Tsetlin, M. L. (1950). Finite-dimensional representations of the group of unimodular matrices. *Doklady Akademii Nauk SSSR*, 71, 825–828.
- Grabmeier, J., & Kerber, A. (1987). The evaluation of irreducible polynomial representations of the general linear groups and of the unitary groups over fields of characteristic 0. *Acta Applicandae Mathematicae*, 8(3), 271–291. <https://doi.org/10.1007/bf00046717>
- Gupt, B., Izaac, J., & Quesada, N. (2019). The Walrus: A library for the calculation of hafnians, Hermite polynomials and Gaussian boson sampling. *Journal of Open Source Software*, 4(44), 1705. <https://doi.org/10.21105/joss.01705>

- Heurtel, N., Fyrrillas, A., Gliniasty, G. de, & others. (2023). Perceval: A software platform for discrete variable photonic quantum computing. *Quantum*, 7, 931. <https://doi.org/10.22331/q-2023-02-21-931>
- Otto, F., & Szymański, K. (2024). Rotational covariance restricts available quantum states. *Phys. Rev. A*, 110(2), 022440. <https://doi.org/10.1103/PhysRevA.110.022440>
- Pawela, Ł., & Puchała, Z. (2026). *IntegrateUnitary.jl: A Julia package for symbolic integration over Haar measures*. <https://arxiv.org/abs/2605.23830>
- Prakash, J. (1996). Wigner's D-matrix elements for SU(3)-a generating function approach. *Journal of Physics A: Mathematical and General*, 29(18), 6059–6072. <https://doi.org/10.1088/0305-4470/29/18/032>
- QuantumKitHub contributors. (2020). *SUNRepresentations.jl*. <https://github.com/QuantumKitHub/SUNRepresentations.jl>.
- Rosset, D., Montealegre-Mora, F., & Bancal, J.-D. (2021). *RepLAB: A computational/numerical approach to representation theory*. <https://github.com/replab/replab>; Springer International Publishing. https://doi.org/10.1007/978-3-030-55777-5_60
- Seron, B., & Restivo, A. (2024). BosonSampling.jl: A Julia package for quantum multi-photon interferometry. *Quantum*, 8, 1378. <https://doi.org/10.22331/q-2024-06-18-1378>